

Activities 1 and 2 – Spark Structured Streaming

Activity 1 – Understanding Spark Execution

Setup

The Spark Structured Streaming application was executed inside Docker using a standalone Spark cluster. Kafka was used as the streaming source, and a load generator continuously produced log records into a Kafka topic.

The application was submitted from the spark-client container using `spark-submit`. Monitoring was done through the Spark Driver Web UI at <http://localhost:4040>.

Execution Behavior

Once the load generator was running, the Structured Streaming tab showed an average input rate of around 8,300 records per second and a processing rate of around 7,300 records per second. This indicated that the application was slightly lagging behind under the baseline configuration.

Jobs and DAG

Each micro-batch triggered a Spark job.

From the DAG visualization, two stages were observed:

- Stage 1 handled Kafka reading, filtering, and projection
- Stage 2 handled shuffle and state operations

This confirms that simple transformations stay within one stage, while stateful operations introduce shuffle boundaries.

Bottleneck Identification

The Stages tab showed that the shuffle stage had the highest execution time.

Shuffle operations and state management were the main performance bottlenecks.

Executors and Resources

Only one executor with one core was used.

CPU parallelism was very limited, and memory usage was low, showing that hardware resources were underutilized.

Activity 1 Conclusion

Spark Structured Streaming processes data in micro-batches composed of jobs and stages.

Shuffle-heavy stages dominate execution time, and the baseline configuration suffered from insufficient parallelism.

Activity 2 – Performance Tuning

Baseline Limitation

The baseline configuration used:

- 1 executor
- 1 core
- 1 GB memory
- Default shuffle partitions

This setup could not fully utilize the available hardware.

Tuning Changes

The application was tuned by:

- Increasing the number of executors
- Increasing executor cores
- Adjusting shuffle partitions

Observed Results

After tuning:

- The processing rate matched or exceeded the input rate
- Executors showed better CPU utilization
- Shuffle stages completed faster
- Executors were often idle between micro-batches, indicating sufficient capacity

Activity 2 Conclusion

Increasing parallelism significantly improved throughput and scalability.

Proper tuning allowed the application to efficiently process the incoming data stream without lag.

Issues Encountered

Several issues were encountered during execution:

- Initially, input and processing rates were zero because the load generator was not running
- The Spark Master UI on port 8080 was misleading, while the Driver UI on port 4040 showed correct streaming metrics
- Some container management issues required restarting jobs with correct configurations

These issues did not affect final results and provided practical insight into real-world Spark deployments.

Bottleneck

The shuffle and state management stage had the longest duration due to data redistribution across executors.

Resource Usage

Memory usage was low in the baseline configuration, while CPU resources were underutilized due to limited parallelism.

Performance and Scalability

Spark Structured Streaming scales by increasing executors, cores, and partitions.

Higher parallelism reduces shuffle impact and improves throughput while maintaining low micro-batch latency.